

Configuration

The configuration module is module ID 6. It records target GPU settings, applies them, reports current and target state, and can re-enforce a target after GPU state changes. The module is loaded when configuration functionality is first requested.

For the properties that can be configured, their valid values, support queries, and target-state lifetime, see [dcgmi config](#).

When configuration is useful

Use the configuration module when a node or workload needs a known GPU runtime state rather than merely an observation of the current state. Typical examples are selecting exclusive-process compute mode for a batch allocation, applying a site power cap, choosing an application-clock pair, enabling ECC where supported, or selecting workload power profiles.

Configuration changes affect the device and potentially every process using it. Apply them through scheduler prologue/epilogue logic or another authority that knows whether the GPU is idle and who owns it. DCGM groups select the target GPUs but do not reserve them, prevent another administrator from changing them, or make a multi-GPU change transactional.

The module maintains two views:

- **Target** is the value requested through DCGM and retained by the running host engine.
- **Current** is the value read from the GPU now.

DCGM applies a target when it is set and can re-enforce it after supported state changes. The target is in-memory host-engine state, not a durable site configuration file. A target can also differ from current state because a property is unsupported, pending, externally changed, or failed to apply.

Example: prepare GPUs for an exclusive job

First identify the assigned GPUs and inspect their current state. Suppose a scheduler allocated GPUs 0 and 1:

```
$ dcgmi discovery --list
$ dcgmi group --create job-4815 --add gpu:0,gpu:1
$ dcgmi config --get --group <group-id> --verbose
```

Replace `<group-id>` with the ID printed by the group command. While the GPUs are idle, request exclusive-process compute mode and verify both target and current values:

```
$ dcgmi config --set --group <group-id> --compmode 2
$ dcgmi config --get --group <group-id> --verbose
```

If another operation reinitializes the GPU while the host engine remains running, reapply the stored target explicitly when needed:

```
$ dcgmi config --enforce --group <group-id>
```

Deleting the group does not undo settings already applied to its GPUs. A job epilogue that must restore unrestricted compute mode should request that state before deleting the group:

```
$ dcgmi config --set --group <group-id> --compmode 0
$ dcgmi group --delete <group-id>
```

Use runtime discovery before setting hardware-dependent properties. For example, `dcgmi discovery --info p --gpuid 0` reports the valid power range, and `dcgmi discovery --info c` reports supported application-clock pairs. A value valid for one member of a heterogeneous group may be invalid for another.

Configuration state and persistence

An important aspect of managing GPUs, especially in multi-node environments, is ensuring consistent configuration across workloads and across devices. In this context the term `configuration` refers to the set of administrative parameters exposed by NVIDIA to tune GPU behavior. DCGM makes it easier for clients to define target configurations and ensure those configurations are maintained over time.

It is important to note that different GPU properties have different levels of persistence. There are two broad categories:

- Device InfoROM lifetime
 - Non-volatile memory on each board, holding certain configurable firmware settings.
 - Persists indefinitely, though firmware can be flashed.
- GPU initialization lifetime
 - Driver level data structures, holding volatile GPU runtime information.
 - Persists until the GPU is de-initialized by the kernel mode driver.

DCGM is primarily focused on maintaining configuration settings that fall into the second category. These settings are normally volatile, potentially resetting each time a GPU becomes idle or is reset. By using DCGM a client can ensure that these settings persist over the desired lifetime.

In most common situations the client should be expected to define a configuration for all GPUs in the system (global group) at system initialization, or define individual partitioned group settings on a per-job basis. Once a configuration has been defined, DCGM applies it and retains the target in the

running host engine. It can restore the target after supported GPU reinitialization events, and clients can request explicit enforcement at job start.

DCGM currently supports the following configuration settings:

Configuration settings		
Setting	Description	Defaults
Sync Boost	Coordinate Auto Boost across GPUs in the group	None
Target Clocks	Attempt to maintain fixed clocks at the target values	None
ECC Mode	Enable ECC protection throughout the GPU's memory	Usually On
Power Limit	Set the maximum allowed power consumption	Varies
Compute Mode	Limit concurrent process access to the GPU	No restrictions
Workload Power Profiles	Enable and disable workload power profiles	None

To define a target configuration for a group, use the `dcgmi config` subcommand. Using the group created in the section above, the following example shows how to set a compute mode target and then list the current configuration state.

```
$ dcgmi config -g 1 --set -c 2
```

```
Configuration successfully set.
```

```
$ dcgmi config -g 1 --get
```

GPU_Group	TARGET CONFIGURATION	CURRENT CONFIGURATION
Group of 2 GPUs		
Sync Boost	Not Specified	Disabled
SM Application Clock	Not Specified	****
Memory Application Clock	Not Specified	****
ECC Mode	Not Specified	****
Power Limit	Not Specified	****
Compute Mode	E. Process	E. Process
Power Profile	Not Specified	Not Specified

**** Non-homogeneous settings across group. Use with -v flag to see details.

```
$ dcgmi config -g 1 --get --verbose
```

GPU ID: 0	TARGET CONFIGURATION	CURRENT CONFIGURATION
Tesla K20c		
Sync Boost	Not Specified	Disabled
SM Application Clock	Not Specified	705
Memory Application Clock	Not Specified	2600
ECC Mode	Not Specified	Disabled
Power Limit	Not Specified	225
Compute Mode	E. Process	E. Process
Power Profile	Not Specified	Not Specified

GPU ID: 1	TARGET CONFIGURATION	CURRENT CONFIGURATION
GeForce GT 430		
Sync Boost	Not Specified	Disabled
SM Application Clock	Not Specified	562
Memory Application Clock	Not Specified	2505
ECC Mode	Not Specified	Enabled
Power Limit	Not Specified	200
Compute Mode	E. Process	E. Process
Power Profile	Not Specified	Not Specified

Once a configuration is set, DCGM maintains the notion of Target and Current state. Target tracks the user's request for configuration state while Current tracks the actual state of the GPU and group. These are generally maintained such that they are equivalent with DCGM restoring current state to target in situations where that state is lost or changed. This can occur after an active diagnostic, a policy action, a GPU reset, or a change made by another management interface.

Workload Power Profile Configuration

The `dcgmi config` command provides functionality to configure and query workload power profiles for NVIDIA GPUs. Workload power profiles allow you to optimize GPU power consumption based on specific workload characteristics.

The supported workload power profiles are specific to the GPU architecture. Use `nvidia-smi power-profiles -l` to see the names and indexes of the supported profiles for the current GPU. The index values used in DCGM are the same as NVML.

Command-line workflow

First inspect the profiles supported by the target GPU. The discovery output distinguishes supported, requested, and currently enforced profiles:

```
$ dcgmi discovery --info w --gpuid 0
```

Then inspect the current and target configuration, apply a profile to the desired GPU group, and verify the result.

1. Check current configuration:

```
$ dcgmi config --get --group 5 --verbose
```

2. Replace the requested profiles with profile 3:

```
$ dcgmi config --set --group 5 --workloadpowerprofile 3 \
  --workloadpowerprofileaction o
```

3. Verify the change:

```
$ dcgmi config --get --group 5 --verbose
```

See [dcgmi config](#) for every configuration property, profile action, special value, support query, and persistence rule. See [dcgmi discovery](#) for workload-profile discovery output. On supported systems, `nvidia-smi power-profiles -l` maps profile indices to names.

Persistence

- Workload power profile settings are applied immediately.
- DCGM retains the target while the host-engine process remains running and can reapply it after GPU reinitialization.
- The target is not durable across a host-engine or system restart.

Troubleshooting

If a profile shows in “Requested” but not “Enforced”:

- The driver may not support the profile on this GPU architecture
- Another power management policy may be taking precedence
- GPU may be in a state that prevents profile changes

If a profile cannot be added, cleared, or enforced, verify the device's supported indices and compare the target and current configuration. The configuration reference documents how to clear one profile or all profiles.

For programmatic access, see the following DCGM API functions:

- `dcgmConfigSetWorkloadPowerProfile()`
- `dcgmGetDeviceWorkloadPowerProfileInfo()`
- `dcgmConfigGet()`

Refer to the DCGM API documentation for detailed function signatures and usage.

Further reading



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.